
boolean



# Gestione Eventi

## in JSX

# Gestione Eventi

## Con HTML + JS

Quando usiamo **HTML** e **JS** gestiamo gli eventi eseguendo alcuni passaggi :

1. Recuperiamo un elemento dal ***DOM***
2. Agganciamo un ***Event Listener***
3. Eseguiamo la sua ***funzione di callback***

```
1 <button>Saluta</button>
```

```
1 const button = document.querySelector('button')
2
3 const sayHello = () => {
4   alert('Ciao!')
5 }
6
7 button.addEventListener('click', sayHello)
```

1
3
2
3

# Gestione Eventi

## JSX

In **JSX**, per catturare gli eventi, usiamo **attributi** che iniziano con "**on**", direttamente sull'***elemento***.

A questi attributi assegniamo una ***funzione di callback*** dichiarata nel componente.

```
1  export default function Counter() {
2
3    const sayHello = () => {
4      alert("ciao");
5    }
6
7    return (
8      <div>
9        <button onClick={sayHello}>Saluta</button>
10     </div>
11   );
12 }
```

Alcuni esempi di attributi per gli eventi:

**onClick**: per catturare il click su un elemento.

**onSubmit**: per gestire il submit di un form.

**onChange**: per reagire ai cambiamenti in un input.

# Gestione Eventi

## Funzioni anonime

Con **HTML + JS** possiamo anche usare una ***funzione anonima*** nell'event listener:

```
1 const button = document.querySelector('button')
2
3 button.addEventListener('click', () => {
4   alert('Ciao!')
5 })
```

Anche in **JSX** è possibile usare una funzione anonima direttamente nell'attributo:

```
1 export default function Counter() {
2
3   return (
4     <div>
5       <button onClick={() => alert("ciao")}>Saluta</button>
6     </div>
7   );
8 }
```



# Concetto di reattività

# Reattività

## Il concetto di "reattività" nel web

Molto spesso, si ha la necessità che ciò che viene mostrato sullo schermo (la UI) si aggiorni a seconda di alcuni eventi che seguono la nostra logica.

In altre parole, la nostra interfaccia utente (UI) deve essere **reattiva**!

*Ad esempio se cliccando un bottone vogliamo incrementare un numero!*



# Reattività

## Con HTML + JS

Quando lavoriamo con HTML e JS, dobbiamo farci carico di tutti gli step necessari per "aggiornare" l'interfaccia utente (UI).

```html
<div>
  <div class="counter">0</div>
  <button>Incrementa!</button>
</div>
```

```js
// Recuperiamo gli elementi
const button = document.querySelector('button');
const counter = document.querySelector('.counter');

// Variabile per contare
let count = 0;

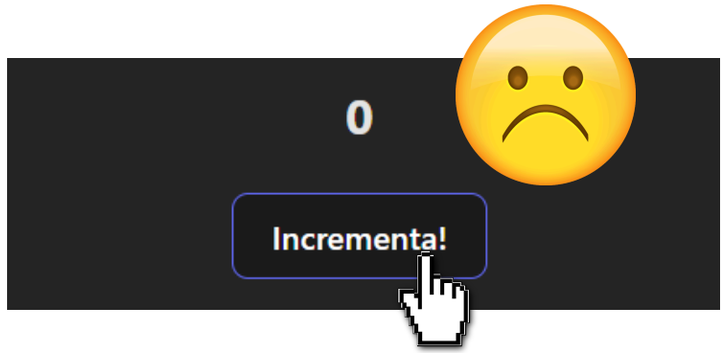
// Gestiamo la reattività
button.addEventListener('click', () => {
  count++;
  counter.innerText = count;
})
```

# Reattività

## Con JSX

In JSX, il nostro approccio è totalmente diverso. Proviamo a creare una variabile ***count*** e ad ***incrementarla*** a seguito di un evento con ***onClick***:

```
1  export default function Counter() {
2
3    let count = 0;
4
5    return (
6      <div>
7        <div class="counter">{count}</div>
8        <button onClick={() => {count++} }>Saluta</button>
9      </div>
10   );
11 }
```




Il valore di count verrà incrementato, ma sulla pagina resterà ancora 0, perchè **la variabile count non è reattiva** e quindi il valore di count a riga 7 non verrà aggiornato sullo schermo.

# Reattività



Ma come faccio a creare una **variabile reattiva**?

# Reattività



Usando lo **Stato** dei componenti!



# Lo Stato

## React e la "reattività" dei componenti

# Lo Stato dei componenti

## useState 🧠

Lo **stato** in React rappresenta le **variabili reattive** di un componente, ossia variabili che possono cambiare valore in futuro.

Quando il valore di una variabile di stato cambia, il componente viene re-renderizzato, aggiornando automaticamente l'interfaccia utente (UI). 🤩

Per creare una variabile di stato usiamo una funzione particolare detta **useState,** in cui inseriamo il valore iniziale della nostra variabile!

```
1 import { useState } from react 👈
2
3 export default function Counter() {
4
5   const mioStato = useState(0) 👈
6
7   //...
8
9 }
```

non dimentichiamo di notare che **useState** va importato! 😉

# Lo Stato dei componenti

## useState 🧠

useState() **restituisce un array** contenente due elementi:

- il **valore attuale** dello stato
- una **funzione** che consente di **aggiornare lo stato**

```
1  import {useState} from react
2
3  export default function Counter() {
4
5    const mioStato = useState(0);
6
7    mioStato[0] // contiene il valore (inizialmente 0);
8
9    mioStato[1] // è una funzione, che mi permetterà di cambiare il valore
10
```



sembra scomodo..

# Lo Stato dei componenti

## useState 🧠

Per comodità è molto meglio usare il **destructuring!**

Per convenzione il **valore** viene chiamato col nome di ciò che questo stato rappresenta (es *count*), mentre la **funzione** viene chiamata con il nome del valore + *set* davanti

```
1 import {useState} from react
2
3 export default function Counter() {
4
5   const [count, setCount] = useState(0);
6
7   //...
8
9 }
```



molto meglio!



# Eventi e Stato

# Lo Stato dei componenti

## Ora si fa sul serio!

Ora che abbiamo importato **useState** e abbiamo ricavato la funzione **setCount**, possiamo usarla in un **onClick** per aggiornare magicamente la UI al cambiamento di **count**!



```jsx
import { useState } from 'react';

export default function Counter() {
  // Definiamo uno stato "count"
  // e una funzione "setCount" per aggiornarlo
  const [count, setCount] = useState(0);

  return (
    <div>
      <div className="counter">{count}</div>
      <button onClick={() => {setCount(count + 1)}}>Incrementa!</button>
    </div>
  );
}
```

# Lo Stato dei componenti

## Occhio alle trappole!

Potremmo avere la tentazione di incrementare count con `count++` ma se lo facessimo, la UI non verrebbe aggiornata!

Non dimentichiamo che scrivere `count++` equivale a scrivere `count = count + 1.`

Ma le variabili di stato **vanno aggiornate soltanto usando la loro funzione!** (quella che inizia con "set" 😉)



```
1   return (
2     <div>
3       <div className="counter">{count}</div>
4       <button onClick={() => {setCount(count++)}}>Incrementa!</button>
5     </div>
6   );
```





# React Hooks

## intro

# React Hooks

## Cosa sono i React hooks?

I React hooks sono delle **funzioni speciali** forniteci da React, che ci permettono di gestire moltissimi aspetti delle nostre applicazioni.

Ce ne sono molti ed ognuno ha il suo scopo. Possiamo anche creare degli **hook custom** ossia delle funzioni personalizzate da poter riutilizzare nei nostri progetti.

Tra quelli principali forniti da React, spiccano certamente **useState** e **useEffect.**

Per utilizzarli, basta importarli dalla libreria React in cima ai componenti

```
1 import {useState, useEffect} from 'react';
```

# React Hooks

## Alcune regole

Quando si utilizzano i **React Hook** è importante tenere a mente queste regole:

- **Nomenclatura**: Tutti gli hooks, compresi quelli che creiamo noi, devono iniziare con "**use**".
- **Convenzione di Nomenclatura**: Per utilizzare un React Hook, il nome del nostro componente deve iniziare con lettera maiuscola.
- **Importazione** per poter utilizzare un hook bisogna prima importarlo nel componente.,
- **Utilizzo nel Componente**: Gli hooks possono essere utilizzati solo all'interno della funzione che definisce il componente.
- **Dichiarazione Condizionale**: Non è possibile dichiarare gli hooks all'interno di condizioni.

# React Hooks

## useState hook

L'hook **useState** ci permette di dichiarare una variabile "dinamica" (detta **variabile di stato**) che React terrà particolarmente d'occhio, cosicchè se il suo valore dovesse cambiare, il componente (ed eventualmente la sua UI)cambierà e si aggiornerà di conseguenza!

**useState** ha le seguenti regole:

- si **invoca** passandole il **valore iniziale** della variabile
- si **destruttura** con un array i cui due elementi sono rispettivamente:
    - il **nome della variabile** (es: `count`)
    - la **funzione** da utilizzare **per modificarne il valore** (es: `setCount`)

```
1 import {useState} from 'react';
2
3 function App(){
4   const [count, setCount] = useState(0);
5 }
6
7 export default App
```



# LIVE CODING

Creiamo il nostro contatore

# ESERCIZIO